

Éditeur de Configuration MIDI pour Ableton Live (.als)

ARCHITECTURE LOGICIELLE & SPÉCIFICATIONS CROSS-PLATFORM (PYTHON)

Ce document dresse l'architecture technique et conceptuelle d'une application de bureau autonome (Standalone) moderne et graphique. Développée en Python pour garantir une portabilité totale entre macOS et Windows, cette application offre aux producteurs et musiciens live une interface visuelle pour manipuler, éditer et appliquer des templates de contrôleurs MIDI directement sur les fichiers de projet Ableton Live (.als), sans altérer les sessions originales.

1. Architecture Globale et Portabilité (UI/UX)

L'application utilise un framework graphique moderne et performant assurant un rendu visuel natif mais stylisé sur les deux systèmes d'exploitation cibles :

- **Framework UI** : CustomTkinter ou PyQt6 avec un thème sombre/lumineux automatique calqué sur l'identité visuelle des stations de travail audionumériques (DAW).
- **Indépendance du Projet** : L'application fonctionne exclusivement par copie mémoire. Le fichier original .als est chargé en lecture seule. Un nouveau fichier de projet est généré uniquement lors de l'exportation explicite par l'utilisateur.
- **Distribution Cross-Platform** : Le code source Python est packagé en exécutables autonomes (.app pour macOS avec signature universelle Apple Silicon/Intel, et .exe pour Windows) via l'outil PyInstaller.

2. Workflow Utilisateur In-App

1. **Chargement** : L'utilisateur clique sur *"Importer un Projet Ableton"*. L'application extrait le fichier .als (GZIP) vers un arbre XML temporaire en mémoire RAM.
2. **Analyse & Visualisation** : Une grille graphique dynamique affiche la structure du projet : Pistes, Effets, Instruments, et statuts des mappings MIDI existants.
3. **Édition & Application de Templates** : L'utilisateur peut manuellement éditer un contrôle (ex: modifier un CC) ou appliquer un *Template de Contrôleur Personnel* (ex: "Mapping Akai APC40 MKII", "Template Arturia KeyLab").
4. **Exportation** : L'utilisateur clique sur *"Régénérer et Exporter"*. Un nouveau fichier .als est compilé, prêt à remplacer l'original ou à être enregistré sous un nouveau nom.

3. Structure Technique du parsing XML Ableton

Le cœur algorithmique repose sur l'identification et la réécriture à chaud des nœuds de configuration de la balise maîtresse <MidiMappingList> d'Ableton Live. Voici la structure XML décodée par l'application pour chaque assignation :

```
<MidiMappingRange>
  <Id Value="102" />
  <MidiControllerChannel Value="0" /> <!-- Canal MIDI : 0 correspond au Canal 1 -->
  <MidiControllerNumber Value="24" /> <!-- Numéro de CC ou Note assignée -->
  <LastTicks Value="0" />
  <RangeTarget>
    <LockableParameter Id="12453" /> <!-- ID unique du paramètre lié dans la session -->
  </RangeTarget>
</MidiMappingRange>
```

4. Spécifications des Fonctionnalités Clés

Module Application	Description Fonctionnelle	Avantage Technique
Gestionnaire de Fichiers Non-Destructif	Mécanisme de sécurité qui duplique le flux binaire lors de l'ouverture d'un fichier .als. L'original reste intact sur le disque dur quoi qu'il arrive dans l'application.	Zéro risque de corruption ou de perte de données sur les projets master de l'artiste.
Moteur de Templates (Profils)	Sauvegarde des configurations de contrôleurs physiques de l'utilisateur sous forme de fichiers de profils légers (.json). Permet de lier instantanément des contrôles par nom ou type.	Possibilité de remapper un projet entier pour un nouveau contrôleur MIDI en un seul clic.
Grille d'Édition Visuelle	Tableau de bord graphique listant les assignations. Tri par piste, filtrage par canal MIDI, modification directe via menus déroulants ou saisie numérique fine.	Plus besoin d'ouvrir le mode de mapping fastidieux d'Ableton. Vue globale de toutes les connexions.

Note Importante sur la Consistance des Identifiants (IDs)

Pour pouvoir "Ajouter" ou "Transférer" des contrôles MIDI d'un projet à un autre, l'application s'assure de faire correspondre les balises <LockableParameter Id="...">. Si la structure des pistes est rigoureusement identique d'une session à l'autre, les templates JSON s'appliqueront instantanément et de façon universelle.

5. Architecture du Code Source Python (Prototype Express)

Le script d'arrière-plan de l'application utilise la bibliothèque standard `gzip` et le parseur haute performance `xml.etree.ElementTree` pour manipuler la session de manière transparente :

```
import gzip
import xml.etree.ElementTree as ET

class AbletonMidiEditor:
    def __init__(self, file_path):
        self.file_path = file_path
        self.tree = None
        self.root = None

    def load_project(self):
        # Lecture sans altérer le fichier d'origine
        with gzip.open(self.file_path, 'rb') as f:
            xml_data = f.read()
        self.root = ET.fromstring(xml_data)
        self.tree = ET.ElementTree(self.root)

    def update_midi_cc(self, mapping_id, new_cc):
        # Recherche et mise à jour dynamique du CC dans le XML
        for mapping in self.root.iter('MidiMappingRange'):
            id_elem = mapping.find('Id')
            if id_elem is not None and id_elem.get('Value') == str(mapping_id):
                cc_elem = mapping.find('MidiControllerNumber')
                if cc_elem is not None:
                    cc_elem.set('Value', str(new_cc))

    def export_project(self, output_path):
        # Régénération par compression GZIP propre
        xml_str = ET.tostring(self.root, encoding='utf-8')
        with gzip.open(output_path, 'wb') as f:
            f.write(xml_str)
```